



404 Not Found

Truong Minh Hieu
Nguyen Tri Khoa
Le Nguyen Quoc Binh

2026-10-08

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

Contest Template

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define ull unsigned long long
#define pii pair<int,int>
#define pll pair<long long, long long>
#define fi first
#define se second
#define all(a) (a).begin(), (a).end()
#define sz(a) (int)(a).size()
#define pb push_back
#define lwb lower_bound
#define upb upper_bound
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
}
```

Shell Setup

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
-fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =<
```

Vim Setup

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on | im jk <esc> | im kj <esc> | no ; :
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \ | tr -d '[:space:]' \
\ | md5sum \ | cut -c-6
```

Code Checksum

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum | cut -c-6
```

Troubleshooting Checklist

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
```

```
Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).
```

```
Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?
```

```
Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned} \Rightarrow$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c
Semiperimeter: $p = \frac{a + b + c}{2}$
Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$
Circumradius: $R = \frac{abc}{4A}$
Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$
Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

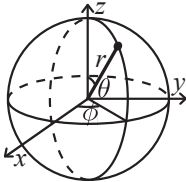
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° ,
 $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx} \tan x = 1 + \tan^2 x \qquad \frac{d}{dx} \arctan x = \frac{1}{1+x^2}$$

$$\int \tan ax = -\frac{\ln |\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) \qquad \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \cdots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

Ordered Set (PBDS)

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. **Time:** $\mathcal{O}(\log N)$

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template <class T>
using OrderedSet = tree<T, null_type, less<T>, rb_tree_tag,
                        tree_order_statistics_node_update>;

void example() {
    OrderedSet<int> t, t2;
    t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

Custom Hash Map

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

```
6827f4, 7 lines

#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct CustomHash { // large odd number for C
    const uint64_t C = (11)(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x * C);
    }
};

__gnu_pbds::gp_hash_table<ll, int, CustomHash> h({}, {}, {}, {}
, {1 << 16});

625ce1, 43 lines
```

```
struct SegTree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    int n;
    vector<T> seg;
    SegTree(int n = 0, T def = unit) : n(n), seg(4 * n + 4, unit)
    {
        if (n > 0) build(1, 1, n, def);
    }
    T merge(T a, T b) const { return max(a, b); }
    void build(int id, int lo, int hi, T val) {
        if (lo == hi) {
            seg[id] = val;
            return;
        }
        int mid = (lo + hi) / 2;
        build(id * 2, lo, mid, val);
        build(id * 2 + 1, mid + 1, hi, val);
        seg[id] = merge(seg[id * 2], seg[id * 2 + 1]);
    }
    void update(int id, int lo, int hi, int pos, T val) {
        if (lo == hi) {
            seg[id] = val;
            return;
        }
        int mid = (lo + hi) / 2;
        if (pos <= mid)
            update(id * 2, lo, mid, pos, val);
        else
            update(id * 2 + 1, mid + 1, hi, pos, val);
        seg[id] = merge(seg[id * 2], seg[id * 2 + 1]);
    }
    void update(int pos, T val) { update(1, 1, n, pos, val); }
    T query(int id, int lo, int hi, int l, int r) const {
        if (r < lo || hi < l) return unit;
        if (l <= lo && hi <= r) return seg[id];
        int mid = (lo + hi) / 2;
        return merge(query(id * 2, lo, mid, l, r), query(id * 2 +
            1, mid + 1, hi, l, r));
    }
    T query(int l, int r) const {
        if (n == 0 || l > r) return unit;
        return query(1, l, n, l, r);
    }
};
```

Segment Tree

Description: Recursive max segment tree. Positions are 1-based; query(l,r) includes both endpoints. update(pos,val) assigns a[pos]. Initially all positions equal def. To change the aggregate, edit T, unit and merge; unit must be its identity.

Usage: SegTree seg(n); seg.update(3, 5); int ans = seg.query(1, 3);

Time: $\mathcal{O}(\log N)$ per operation; $\mathcal{O}(N)$ memory.

Lazy Segment Tree

Description: Array-based segment tree for range assignment, range addition and range maximum. Positions are 1-based; all ranges [l,r] are inclusive. Values and additions must fit int. Vector constructor uses a[1..n] (a[0] unused). Initial value defaults to -inf. hasSet distinguishes a pending assignment from no assignment, so every int value is allowed.

Usage: LazySegTree seg(n, 0); seg.add(1,n,3); seg.set(2,4,5); int ans = seg.query(1,n);

Time: $\mathcal{O}(\log N)$ per operation; $\mathcal{O}(N)$ memory.

```
7af884, 84 lines

const int inf = 1000000000;
struct LazySegTree {
    struct Node {
        int val = -inf;
        int lazySet = 0;
        int lazyAdd = 0;
        bool hasSet = false;
    };
    int n;
    vector<Node> seg;
    LazySegTree(int n, int initial = -inf) : n(n), seg(4 * n + 4,
        Node{initial}) {}
    LazySegTree(const vector<int> &a) : LazySegTree(max(0, sz(a)
        - 1)) {
        if (n > 0) build(1, 1, n, a);
    }
    void build(int id, int lo, int hi, const vector<int> &a) {
        if (lo == hi) {
            seg[id].val = a[lo];
            return;
        }
        int mid = (lo + hi) / 2;
        build(id * 2, lo, mid, a);
        build(id * 2 + 1, mid + 1, hi, a);
        pull(id);
    }
    void pull(int id) { seg[id].val = max(seg[id * 2].val, seg[id
        * 2 + 1].val); }
    void applySet(int id, int val) {
        seg[id].val = val;
        seg[id].lazySet = val;
        seg[id].lazyAdd = 0;
        seg[id].hasSet = true;
    }
    void applyAdd(int id, int val) {
        seg[id].val += val;
        if (seg[id].hasSet)
            seg[id].lazySet += val;
        else
            seg[id].lazyAdd += val;
    }
    void push(int id) {
        // Assignment overwrites older updates; addition comes
        // after assignment.
        if (seg[id].hasSet) {
            applySet(id * 2, seg[id].lazySet);
            applySet(id * 2 + 1, seg[id].lazySet);
            seg[id].hasSet = false;
        }
        if (seg[id].lazyAdd != 0) {
            applyAdd(id * 2, seg[id].lazyAdd);
            applyAdd(id * 2 + 1, seg[id].lazyAdd);
            seg[id].lazyAdd = 0;
        }
    }
    void update(int id, int lo, int hi, int l, int r, int val,
        bool assign) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) {
```

```
        if (assign)
            applySet(id, val);
        else
            applyAdd(id, val);
        return;
    }
    push(id);
    int mid = (lo + hi) / 2;
    update(id * 2, lo, mid, l, r, val, assign);
    update(id * 2 + 1, mid + 1, hi, l, r, val, assign);
    pull(id);
}
void set(int l, int r, int val) {
    if (n > 0 && l <= r) update(l, l, n, l, r, val, true);
}
void add(int l, int r, int val) {
    if (n > 0 && l <= r) update(l, l, n, l, r, val, false);
}
int query(int id, int lo, int hi, int l, int r) {
    if (r < lo || hi < l) return INT_MIN;
    if (l <= lo && hi <= r) return seg[id].val;
    push(id);
    int mid = (lo + hi) / 2;
    return max(query(id * 2, lo, mid, l, r), query(id * 2 + 1,
        mid + 1, hi, l, r));
}
int query(int l, int r) {
    if (n == 0 || l > r) return INT_MIN;
    return query(l, l, n, l, r);
}
};
```

Rollback DSU

Description: Disjoint-set data structure with undo. If undo is not needed, skip `st, time()` and `rollback()`.
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log N)$

640d1a, 23 lines

```
struct RollbackDSU {
    vector<int> e;
    vector<pii> st;
    RollbackDSU(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i-- > t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};
```

2D Prefix Sums

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: `PrefixSum2D<int> m(matrix);`
`m.sum(0, 0, 2, 2);` // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

c580ca, 14 lines

```
template <class T>
struct PrefixSum2D {
    vector<vector<T>>> p;
    PrefixSum2D(vector<vector<T>>> &v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R + 1, vector<T>(C + 1));
        for (int r = 0; r < (R); ++r)
            for (int c = 0; c < (C); ++c)
                p[r + 1][c + 1] = v[r][c] + p[r][c + 1] + p[r + 1][c] -
                    p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

Matrix

Description: Basic operations on square matrices.

Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}}, {{4,5,6}}, {{7,8,9}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

af2afd, 29 lines

```
template <class T, int N>
struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M &m) const {
        M a;
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j)
                for (int k = 0; k < (N); ++k) a.d[i][k] += d[i][j] * m.
                    d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N> &vec) const {
        array<T, N> ret{};
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);
        for (int i = 0; i < (N); ++i) a.d[i][i] = 1;
        while (p) {
            if (p & 1) a = a * b;
            b = b * b;
            p >>= 1;
        }
        return a;
    }
};
```

Convex Hull Trick (CHT)

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

dced59, 32 lines

```
struct CHTLine {
    mutable ll k, m, p;
    bool operator<(const CHTLine &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct CHT : multiset<CHTLine, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
```

```
ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b);
}
bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k)
        x->p = x->m > y->m ? inf : -inf;
    else
        x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
}
void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x->p >= y->p)
        isect(x, erase(y)));
}
ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
}
};
```

Implicit Treap

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

994977, 55 lines

```
struct TreapNode {
    TreapNode *l = 0, *r = 0;
    int val, y, c = 1;
    TreapNode(int val) : val(val), y(rand()) {}
    void recalc();
};
int cnt(TreapNode *n) { return n ? n->c : 0; }
void TreapNode::recalc() { c = cnt(l) + cnt(r) + 1; }
template <class F>
void each(TreapNode *n, F f) {
    if (n) {
        each(n->l, f);
        f(n->val);
        each(n->r, f);
    }
}
pair<TreapNode *, TreapNode *> split(TreapNode *n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    } else {
        auto [L, R] = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = L;
        n->recalc();
        return {n, R};
    }
}
TreapNode *merge(TreapNode *l, TreapNode *r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        return l->recalc(), l;
    } else {
        r->l = merge(l, r->l);
        return r->recalc(), r;
    }
}
```

```
        return r->recalc(), r;
    }
}

TreapNode *ins(TreapNode *t, TreapNode *n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

// Example application: move the range [l, r) to index k
void move(TreapNode *&t, int l, int r, int k) {
    TreapNode *a, *b, *c;
    tie(a, b) = split(t, l);
    tie(b, c) = split(b, r - 1);
    if (k <= l)
        t = merge(ins(a, b, k), c);
    else
        t = merge(a, ins(c, b, k - r));
}
```

Fenwick Tree (BIT)

Description: 1-based Fenwick tree. update(pos, delta) adds to a[pos]; query(pos) sums [1, pos]. lowerBound requires nonnegative elements; returns 0 for sum <= 0, or n+1 if absent.
Usage: Fenwick bit(n); bit.update(3, 5); ll sum = bit.query(3);
Time: $\mathcal{O}(\log N)$ per operation; $\mathcal{O}(N)$ memory.

```
76d6e4, 32 lines
struct Fenwick {
    int n;
    vector<ll> bit;
    Fenwick(int n = 0) : n(n), bit(n + 1, 0) {}
    void update(int pos, ll delta) {
        for (int i = pos; i <= n; i += i & -i) bit[i] += delta;
    }
    ll query(int pos) const {
        ll sum = 0;
        for (int i = pos; i > 0; i -= i & -i) sum += bit[i];
        return sum;
    }
    ll query(int l, int r) const {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
    int lowerBound(ll sum) const {
        if (sum <= 0) return 0;
        int pos = 0;
        int step = 1;
        while (step <= n / 2) step *= 2;
        // pos is the last prefix whose sum is smaller than the
        // target.
        for (; step > 0; step /= 2) {
            int next = pos + step;
            if (next <= n && bit[next] < sum) {
                pos = next;
                sum -= bit[next];
            }
        }
        return pos + 1;
    }
};
```

2D Fenwick Tree (BIT)

Description: Offline 2D Fenwick tree. x is in [1, n]; y can be any integer. Register every update coordinate with fakeUpdate before init. query(x,y) sums points with first coordinate <= x and second coordinate <= y. Construct a new object to reset.
Usage: Fenwick2D bit(n); bit.fakeUpdate(x,y); bit.init(); bit.update(x,y,5); bit.query(x,y);

```
Time:  $\mathcal{O}(\log^2 N)$  per operation;  $\mathcal{O}(U \log N)$  memory for U registered points.
"FenwickTree.h"
aaf063, 30 lines
struct Fenwick2D {
    int n;
    vector<vector<int>>> ys;
    vector<Fenwick> bit;
    Fenwick2D(int n) : n(n), ys(n + 1), bit(n + 1) {}
    void fakeUpdate(int x, int y) {
        for (int i = x; i <= n; i += i & -i) ys[i].push_back(y);
    }
    void init() {
        for (int i = 1; i <= n; i++) {
            sort(all(ys[i]));
            ys[i].erase(unique(all(ys[i])), ys[i].end()));
            bit[i] = Fenwick(sz(ys[i]));
        }
    }
    void update(int x, int y, ll delta) {
        for (int i = x; i <= n; i += i & -i) {
            int pos = lower_bound(all(ys[i]), y) - ys[i].begin() + 1;
            bit[i].update(pos, delta);
        }
    }
    ll query(int x, int y) const {
        ll sum = 0;
        for (int i = x; i > 0; i -= i & -i) {
            int pos = upper_bound(all(ys[i]), y) - ys[i].begin();
            sum += bit[i].query(pos);
        }
        return sum;
    }
};
```

Sparse Table (RMQ)

Description: Range Minimum Queries on an array. Returns min(V[a, V[a + 1], ... V[b - 1]) in constant time.
Usage: SparseTable rmq(values); rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
4eddea, 16 lines
template <class T>
struct SparseTable {
    vector<vector<T>>> jmp;
    SparseTable(const vector<T> &V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            for (int j = 0; j < (sz(jmp[k])); ++j)
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

Mo’s Algorithm

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a,c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

```
2c5c29, 49 lines
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc(){...} // compute current answer
```

```
vector<int> mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; //  $\sim N/\sqrt{Q}$ 
    vector<int> s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t) { return K(Q[s]) < K(Q[t]); })
    ;
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
    return res;
}

vector<int> moTree(vector<array<int, 2>> Q, vector<vector<int>>>
    &ed, int root = 0) {
    int N = sz(ed), pos[2] = {}, blk = 350; //  $\sim N/\sqrt{Q}$ 
    vector<int> s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N
        );
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto &f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x])
            if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
    #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t) { return K(Q[s]) < K(Q[t]); })
    ;
    for (int qi : s)
        for (int end = 0; end < (2); ++end) {
            int &a = pos[end], b = Q[qi][end], i = 0;
        #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
            else { add(c, end); in[c] = 1; } a = c; }
            while (!L[b] <= L[a] && R[a] <= R[b]))
                I[i++] = b, b = par[b];
            while (a != b) step(par[a]);
            while (i--) step(I[i]);
            if (end) res[qi] = calc();
        }
    return res;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial

```
f221fd, 18 lines
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        for (int i = 1; i < (sz(a)); ++i) a[i - 1] = i * a[i];
        a.pop_back();
    }
};
```

```
void divroot(double x0) {
    double b = a.back(), c;
    a.back() = 0;
    for (int i = sz(a) - 1; i--;) c = a[i], a[i] = a[i + 1] *
        x0 + b, b = c;
    a.pop_back();
}
};
```

Polynomial Roots

Description: Finds the real roots to a polynomial.
Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h" a0e4b3, 27 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) {
        return {-p.a[0] / p.a[1]};
    }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin - 1);
    dr.push_back(xmax + 1);
    sort(all(dr));
    for (int i = 0; i < (sz(dr) - 1); ++i) {
        double l = dr[i], h = dr[i + 1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            for (int it = 0; it < (60); ++it) { // while (h - l > 1e
                -8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign)
                    l = m;
                else
                    h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
    return ret;
}
```

Polynomial Interpolation

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k / (n - 1) * \pi), k = 0 \dots n - 1$.
Time: $\mathcal{O}(n^2)$

c0e0f5, 16 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    for (int k = 0; k < (n - 1); ++k)
        for (int i = k + 1; i < (n); ++i)
            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0;
    temp[0] = 1;
    for (int k = 0; k < (n); ++k)
        for (int i = 0; i < (n); ++i) {
            res[i] += y[k] * temp[i];
            swap(last, temp[i]);
            temp[i] -= last * x[k];
        }
    return res;
}
```

Linear Recurrence

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i - j - 1]tr[j]$, given $S[0 \dots \geq n - 1]$ and $tr[0 \dots n - 1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

500941, 28 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        for (int i = 0; i < (n + 1); ++i)
            for (int j = 0; j < (n + 1); ++j)
                res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i)
            for (int j = 0; j < (n); ++j)
                res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
                    mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    for (int i = 0; i < (n); ++i) res = (res + pol[i + 1] * S[i])
        % mod;
    return res;
}
```

4.2 Matrices

Determinant

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

5b9f27, 18 lines

```
double det(vector<vector<double>>> &a) {
    int n = sz(a);
    double res = 1;
    for (int i = 0; i < (n); ++i) {
        int b = i;
        for (int j = i + 1; j < (n); ++j)
            if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        for (int j = i + 1; j < (n); ++j) {
            double v = a[j][i] / a[i][i];
            if (v != 0)
                for (int k = i + 1; k < (n); ++k) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

Integer Determinant

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

958a70, 20 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>> &a) {
    int n = sz(a);
    ll ans = 1;
    for (int i = 0; i < (n); ++i) {
        for (int j = i + 1; j < (n); ++j) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t)
                    for (int k = i; k < (n); ++k)
                        a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

Gaussian Elimination

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2 m)$

ae7962, 40 lines

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd> &A, vd &b, vd &x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vector<int> col(m);
    iota(all(col), 0);

    for (int i = 0; i < (n); ++i) {
        double v, bv = 0;
        for (int r = i; r < (n); ++r)
            for (int c = i; c < (m); ++c)
                if ((v = fabs(A[r][c])) > bv)
                    br = r, bc = c, bv = v;
        if (bv <= eps) {
            for (int j = i; j < (n); ++j)
                if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < (n); ++j) swap(A[j][i], A[j][bc]);
        bv = 1 / A[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            for (int k = i + 1; k < (m); ++k) A[j][k] -= fac * A[i][k];
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        for (int j = 0; j < (i); ++j) b[j] -= A[j][i] * b[i];
    }
}
```

```
    }
    return rank; // (multiple solutions if rank < m)
}
```

Gaussian Elimination: Unique Values

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h"	0d7689, 10 lines
-----------------	------------------

```
for (int j = 0; j < (n); ++j)
    if (j != i) // instead of for (int j = i+1; j < (n); ++j)
        // ... then at the end:
        x.assign(m, undefined);
for (int i = 0; i < (rank); ++i) {
    for (int j = rank; j < (m); ++j)
        if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:;
}
```

Gaussian Elimination over GF(2)

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

	bb5caa, 39 lines
--	------------------

```
typedef bitset<1000> bs;
int solveLinear(vector<bs> &A, vector<int> &b, bs &x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vector<int> col(m);
    iota(all(col), 0);
    for (int i = 0; i < (n); ++i) {
        for (br = i; br < n; ++br)
            if (A[br].any()) break;
        if (br == n) {
            for (int j = i; j < (n); ++j)
                if (b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i - 1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < (n); ++j)
            if (A[j][i] != A[j][bc]) {
                A[j].flip(i);
                A[j].flip(bc);
            }
        for (int j = i + 1; j < (n); ++j)
            if (A[j][i]) {
                b[j] ^= b[i];
                A[j] ^= A[i];
            }
        rank++;
    }

    x = bs();
    for (int i = rank; i--; ) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        for (int j = 0; j < (i); ++j) b[j] ^= A[j][i];
    }
    return rank; // (multiple solutions if rank < m)
}
```

Matrix Inverse

Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.

	56a303, 40 lines
--	------------------

```
int matInv(vector<vector<double>> &A) {
    int n = sz(A);
    vector<int> col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    for (int i = 0; i < (n); ++i) tmp[i][i] = 1, col[i] = i;

    for (int i = 0; i < (n); ++i) {
        int r = i, c = i;
        for (int j = i; j < (n); ++j)
            for (int k = i; k < (n); ++k)
                if (fabs(A[j][k]) > fabs(A[r][c]))
                    r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]);
        tmp[i].swap(tmp[r]);
        for (int j = 0; j < (n); ++j)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            for (int k = i + 1; k < (n); ++k) A[j][k] -= f * A[i][k];
            for (int k = 0; k < (n); ++k) tmp[j][k] -= f * tmp[i][k];
        }
        for (int j = i + 1; j < (n); ++j) A[i][j] /= v;
        for (int j = 0; j < (n); ++j) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for (int i = n - 1; i > 0; --i)
        for (int j = 0; j < (i); ++j) {
            double v = A[j][i];
            for (int k = 0; k < (n); ++k) tmp[j][k] -= v * tmp[i][k];
        }

    for (int i = 0; i < (n); ++i)
        for (int j = 0; j < (n); ++j) A[col[i]][col[j]] = tmp[i][j];
    return n;
}
```

Tridiagonal Linear System

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

	a708d7, 28 lines
--	------------------

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T> &super,
                    const vector<T> &sub, vector<T> b) {

    int n = sz(b);
    vector<int> tr(n);
    for (int i = 0; i < (n - 1); ++i) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i + 1] -= b[i] * diag[i + 1] / super[i];
            if (i + 2 < n) b[i + 2] -= b[i] * sub[i + 1] / super[i];
            diag[i + 1] = sub[i];
            tr[++i] = 1;
        } else {
            diag[i + 1] -= super[i] * sub[i] / diag[i];
            b[i + 1] -= b[i] * sub[i] / diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i - 1]);
            diag[i - 1] = diag[i];
            b[i] /= super[i - 1];
        } else {
            b[i] /= diag[i];
            if (i) b[i - 1] -= b[i] * super[i - 1];
        }
    }
    return b;
}
```

Number theory (5)

5.1 Modular arithmetic

Modular Arithmetic

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"	ealac7, 21 lines
------------	------------------

```
const ll mod = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll y) : Mod(y % mod + mod, 0) {}
    Mod(ll y, int) : x(y < mod ? y : y - mod) {}
    Mod operator+(Mod b) { return {x + b.x, 0}; }
    Mod operator-(Mod b) { return {x - b.x + mod, 0}; }
    Mod operator*(Mod b) { return {x * b.x % mod, 0}; }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1);
        return x;
    }
    Mod operator^(ll e) {
        if (!e) return 1;
        Mod r = *this ^ (e / 2);
        r = r * r;
        return e & 1 ? *this * r : r;
    }
};
```

Modular Inverses

Description: Pre-computation of modular inverses. Assumes $\text{LIM} \leq \text{mod}$ and that mod is a prime.

	a1a002, 4 lines
--	-----------------

```
const ll mod = 1000000007, LIM = 200000;
ll *inv = new ll[LIM] - 1;
inv[1] = 1;
```

```
for (int i = 2; i < (LIM); ++i) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

Modular Exponentiation

b83e45, 7 lines

```
const ll mod = 1000000007; // faster if const
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

Discrete Logarithm

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$ ea7d02, 12 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        for (int i = 2; i < (n + 2); ++i)
            if (A.count(e = e * f % m))
                return n * i - A[e];
    return -1;
}
```

Floor Sum and Modular Sum

Description: Sums of mod'ed arithmetic progressions.

modsum(to, c, k, m) = $\sum_{i=0}^{to-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.670e3d, 15 lines

```
ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m;
    c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

64-bit Modular Multiplication

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

c52063, 10 lines

```
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * (ull)(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}

ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

Modular Square Root

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h"19a793, 25 lines

```
ll sqrt(ll a, ll p) {
    a %= p;
    if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p - 1) / 2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p + 1) / 4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (;;) r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

Fast Prime Sieve

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx$ 1.5s

f5c449, 22 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vector<int> eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vector<int> pr = {2}, sieve(S + 1);
    pr.reserve(int(LIM / log(LIM) * 1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2)
        if (!sieve[i]) {
            cp.push_back({i, i * i / 2});
            for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
        }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i = idx; i < S + L; idx = (i += p)) block[i - L] = 1;
        for (int i = 0; i < (min(S, R - L)); ++i)
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

Miller-Rabin Primality Test

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

"ModMulLL.h"60dcd1, 12 lines

```
bool isPrime(ull n) {
```

```
if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
for (ull a : A) { // ^ count trailing zeroes
    ull p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
        p = modmul(p, p, n);
    if (p != n - 1 && i != s) return 0;
}
return 1;
}
```

Pollard-Rho Factorization

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"d8d98d, 18 lines

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

Extended Euclidean Algorithm

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a / b * x, d;
}
```

Chinese Remainder Theorem (CRT)

Description: Chinese Remainder Theorem.

crt(a, m, b, n) computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

"euclid.h"04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{gcd(a,b)}, y - \frac{ka}{gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

Euler’s Totient Function

Description: *Euler’s ϕ function* is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

```
3a1929, 8 lines
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
    for (int i = 0; i < (LIM); ++i) phi[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < LIM; i += 2)
        if (phi[i] == i)
            for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

Continued Fractions

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $O(\log N)$

```
dd6c5e, 23 lines
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX;
    d y = x;
    for (;) {
        ll lim = min(P ? (N - LP) / P : inf, Q ? (N - LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b * P + LP, NQ = b * Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1 / (y - (d)a)) > 3 * N) {
            return {NP, NQ};
        }
        LP = P;
        P = NP;
        LQ = Q;
        Q = NQ;
    }
}
```

Fraction Binary Search

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: fracBS({}(Frac f) { return f.p>=3*f.q; }, 10); // {1, 3}
Time: $O(\log(N))$

```
27ab3e, 28 lines
struct Frac {
    ll p, q;
};
template <class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step;
                si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B;
        B = !adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

Permutation Ranking

Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $O(n)$

```
7ad253, 6 lines
int permToInt(vector<int> &v) {
    int use = 0, i = 0, r = 0;
    for (int x : v) r = r * ++i + __builtin_popcount(use & -(1 << x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

Multinomial Coefficients

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$. 5f59d2, 6 lines

```
11 multinomial(vector<int>& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    for (int i = 1; i < (sz(v)); ++i)
        for (int j = 0; j < (v[i]); ++j) c = c * ++m / (j + 1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^\infty f(i) = \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m)$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n - 1, k - 1) + (n - 1) c(n - 1, k), \quad c(0, 0) = 1$$

$$\sum_{k=0}^n c(n, k) x^k = x(x + 1) \dots (x + n - 1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$

$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j + 1)$, $k + 1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n - k) E(n - 1, k - 1) + (k + 1) E(n - 1, k)$$

$$E(n, 0) = E(n, n - 1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n - 1, k - 1) + k S(n - 1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) =$

1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, $\dots$. For p prime,

$$B(p^m + n) \equiv m B(n) + B(n + 1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}

on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$

with degrees d_i : $(n - 2)! / ((d_1 - 1)! \dots (d_n - 1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.

- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

Bellman-Ford Shortest Paths

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get `dist = inf`; nodes reachable through negative-weight cycles get `dist = -inf`. Assumes $V^2 \max |w_i| < \sim 2^{63}$. **Time:** $\mathcal{O}(VE)$ 5440f9, 29 lines

```
const ll inf = LLONG_MAX;
struct BellmanFordEdge {
    int a, b, w, s() { return a < b ? a : -a; }
};
struct BellmanFordState {
    ll dist = inf;
    int prev = -1;
};
void bellmanFord(vector<BellmanFordState>& nodes, vector<
    BellmanFordEdge>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](BellmanFordEdge a, BellmanFordEdge b) {
        return a.s() < b.s(); });

    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
    for (int i = 0; i < (lim); ++i)
        for (BellmanFordEdge ed : eds) {
            BellmanFordState cur = nodes[ed.a], &dest = nodes[ed.b];
            if (abs(cur.dist) == inf) continue;
            ll d = cur.dist + ed.w;
            if (d < dest.dist) {
                dest.prev = ed.a;
                dest.dist = (i < lim - 1 ? d : -inf);
            }
        }
    for (int i = 0; i < (lim); ++i)
        for (BellmanFordEdge e : eds) {
            if (nodes[e.a].dist == -inf)
                nodes[e.b].dist = -inf;
        }
}
```

Topological Sort

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.

Time: $\mathcal{O}(|V| + |E|)$ 73359c, 11 lines

```
vector<int> topoSort(const vector<vector<int>>& gr) {
    vector<int> indeg(sz(gr)), q;
    for (auto &li : gr)
        for (int x : li) indeg[x]++;
    for (int i = 0; i < (sz(gr)); ++i)
        if (indeg[i] == 0) q.push_back(i);
    for (int j = 0; j < (sz(q)); ++j)
        for (int x : gr[q[j]])
            if (--indeg[x] == 0) q.push_back(x);
    return q;
}
```

```
}


```

7.2 Network flow

Push-Relabel Max Flow

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vector<int>>> hs;
    vector<int> H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2 * n), H(n) {}
    void addEdge(int s, int t, ll cap, ll rcap = 0) {
        if (s == t) return;
        g[s].push_back({t, sz(g[t]), 0, cap});
        g[t].push_back({s, sz(g[s]) - 1, 0, rcap});
    }
    void addFlow(Edge &e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f;
        e.c -= f;
        ec[e.dest] += f;
        back.f -= f;
        back.c += f;
        ec[back.dest] -= f;
    }
    ll calc(int s, int t) {
        int v = sz(g);
        H[s] = v;
        ec[t] = 1;
        vector<int> co(2 * v);
        co[0] = v - 1;
        for (int i = 0; i < (v); ++i) cur[i] = g[i].data();
        for (Edge &e : g[s]) addFlow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty())
                if (!hi--) return -ec[s];
            int u = hs[hi].back();
            hs[hi].pop_back();
            while (ec[u] > 0) // discharge u
                if (cur[u] == g[u].data() + sz(g[u])) {
                    H[u] = 1e9;
                    for (Edge &e : g[u])
                        if (e.c && H[u] > H[e.dest] + 1)
                            H[u] = H[e.dest] + 1, cur[u] = &e;
                    if (++co[H[u]], !--co[hi] && hi < v)
                        for (int i = 0; i < (v); ++i)
                            if (hi < H[i] && H[i] < v)
                                --co[H[i]], H[i] = v + 1;
                    hi = H[u];
                } else if (cur[u]->c && H[u] == H[cur[u]->dest] + 1)
                    addFlow(*cur[u], min(ec[u], cur[u]->c));
                else
                    ++cur[u];
        }
    }
    bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

Min-Cost Max-Flow

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;
struct MinCostMaxFlow {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vector<int> seen;
    vector<ll> dist, pi;
    vector<edge*> par;
    MinCostMaxFlow(int N) : N(N), ed(N), seen(N), dist(N), pi(N),
        par(N) {}
    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }
    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0;
        ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        while (!q.empty()) {
            s = q.top().second;
            q.pop();
            seen[s] = 1;
            di = dist[s] + pi[s];
            for (edge &e : ed[s])
                if (!seen[e.to]) {
                    ll val = di - pi[e.to] + e.cost;
                    if (e.cap - e.flow > 0 && val < dist[e.to]) {
                        dist[e.to] = val;
                        par[e.to] = &e;
                        if (its[e.to] == q.end())
                            its[e.to] = q.push({-dist[e.to], e.to});
                        else
                            q.modify(its[e.to], {-dist[e.to], e.to});
                    }
                }
        }
        for (int i = 0; i < (N); ++i) pi[i] = min(pi[i] + dist[i], INF);
    }
    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge *x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge *x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
```

```
                ed[x->to][x->rev].flow -= fl;
            }
        }
        for (int i = 0; i < (N); ++i)
            for (edge &e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost / 2};
    }
};

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while (ch-- && it--)
        for (int i = 0; i < (N); ++i)
            if (pi[i] != INF)
                for (edge &e : ed[i])
                    if (e.cap)
                        if ((v = pi[i] + e.cost) < pi[e.to])
                            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}

};


```

Edmonds-Karp Max Flow

Description: Flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

```
template <class T>
T edmondsKarp(vector<unordered_map<int, T>> &
               graph,
               int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vector<int> par(sz(graph)), q = par;

    for (;;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        for (int i = 0; i < (ptr); ++i) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
    out:
        T inc = numeric_limits<T>::max();
        for (int y = sink; y != source; y = par[y])
            inc = min(inc, graph[par[y]][y]);

        flow += inc;
        for (int y = sink; y != source; y = par[y]) {
            int p = par[y];
            if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
            graph[y][p] += inc;
        }
    }
};
```

Minimum Cut from Max Flow

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

7.3 Matching

Hopcroft-Karp Matching

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

```
5e382a, 25 lines
int hopcroftKarp(vector<vector<int>> &g, vector<int> &r) {
    int n = sz(g), res = 0;
    vector<int> l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        for (int i = 0; i < (n); ++i)
            if (l[i] == -1) q[t++] = i, d[i] = 1;
        for (int i = 0; i < (t); ++i)
            for (int v : g[q[i]]) {
                if (r[v] == -1)
                    f = 1;
                else if (!d[r[v]])
                    d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
            }
        if (!f) return res;
        for (int i = 0; i < (n); ++i)
            if (l[i] == -1) res += dfs(dfs, i);
    }
}
```

Bipartite Matching (DFS)

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: vector<int> btoa(m, -1); dfsMatching(g, btoa);
Time: $\mathcal{O}(VE)$

```
1eb922, 23 lines
bool find(int j, vector<vector<int>> &g, vector<int> &btoa,
vector<int> &vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1;
    int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}
int dfsMatching(vector<vector<int>> &g, vector<int> &btoa) {
    vector<int> vis;
    for (int i = 0; i < (sz(g)); ++i) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
}
```

```

    }
}
return sz(btoa) - (int)count(all(btoa), -1);
}
```

Bipartite Vertex Cover

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

```
6a41c1, 26 lines
"DFSMatching.h"
vector<int> cover(vector<vector<int>> &g, int n, int m) {
    vector<int> match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match)
        if (it != -1) lfound[it] = false;
    vector<int> q, cover;
    for (int i = 0; i < (n); ++i)
        if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back();
        q.pop_back();
        lfound[i] = 1;
        for (int e : g[i])
            if (!seen[e] && match[e] != -1) {
                seen[e] = true;
                q.push_back(match[e]);
            }
    }
    for (int i = 0; i < (n); ++i)
        if (!lfound[i]) cover.push_back(i);
    for (int i = 0; i < (m); ++i)
        if (seen[i]) cover.push_back(n + i);
    assert(sz(cover) == res);
    return cover;
}
```

Hungarian Algorithm (Assignment)

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j] = cost$ for $L[i]$ to be matched with $R[j]$, and returns (min cost, match), where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

```
97256e, 25 lines
template <class T>
pair<T, vector<int>> weightedMatching(vector<vector<T>> &C) {
    int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
    vector<T> dist(m), pot(m);
    vector<int> match(i), rev(m, -1), cols(m), prev(m);
    T d = 0, nd, cost = 0;
    while (i--) {
        for (int c = 0; c < (m); ++c) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
        for (s = 0;;) {
            for (int j = s; j < (m); ++j) {
                c = cols[j], nd = dist[c] - pot[c];
                if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
            }
            if ((r = rev[c = cols[s++]]) == -1) break;
            for (int j = 0; j < (m); ++j)
                if (dist[j] > (nd = C[r][j] - C[r][c] + dist[c]))
                    dist[j] = nd, prev[j] = r;
        }
        cost += dist[c];
        while (s--) pot[cols[s]] = dist[cols[s]] - d;
        for (; r != i; swap(c, match[r]))
            r = rev[c] = prev[c];
    }
}
```

```

    }
    return {cost, match};
}

General Graph Matching
Description: Matching for general graphs. Fails with probability  $N/mod$ .
Time:  $\mathcal{O}(N^3)$ 
"./numerical/MatrixInverse-mod.h"
809929, 48 lines
vector<pii> generalMatching(int N, vector<pii> &ed) {
    vector<vector<ll>> mat(N, vector<ll>(N)), A;
    for (pii pa : ed) {
        int a = pa.first, b = pa.second, r = rand() % mod;
        mat[a][b] = r, mat[b][a] = (mod - r) % mod;
    }

    int r = matInv(A = mat), M = 2 * N - r, matchI, matchJ;
    assert(r % 2 == 0);

    if (M != N) do {
        mat.resize(M, vector<ll>(M));
        for (int i = 0; i < (N); ++i) {
            mat[i].resize(M);
            for (int j = N; j < (M); ++j) {
                int r = rand() % mod;
                mat[i][j] = r, mat[j][i] = (mod - r) % mod;
            }
        } while (matInv(A = mat) != M);

    vector<int> has(M, 1);
    vector<pii> ret;
    for (int it = 0; it < (M / 2); ++it) {
        for (int i = 0; i < (M); ++i)
            if (has[i])
                for (int j = i + 1; j < (M); ++j)
                    if (A[i][j] && mat[i][j]) {
                        matchI = i;
                        matchJ = j;
                        goto done;
                    }
        assert(0);
    done:
        if (matchJ < N) ret.emplace_back(matchI, matchJ);
        has[matchI] = has[matchJ] = 0;
        for (int sw = 0; sw < (2); ++sw) {
            ll a = modpow(A[matchI][matchJ], mod - 2);
            for (int i = 0; i < (M); ++i)
                if (has[i] && A[i][matchJ]) {
                    ll b = A[i][matchJ] * a % mod;
                    for (int j = 0; j < (M); ++j) A[i][j] = (A[i][j] - A[matchI][j] * b) % mod;
                }
            swap(matchI, matchJ);
        }
    }
    return ret;
}
```

7.4 DFS algorithms

Strongly Connected Components (SCC)

Description: Tarjan's strongly connected components for a directed graph. Default vertices are $1..n$ (adj[0] unused); pass firstVertex=0 for 0-based graphs. comp[u] is the component ID; components[id] contains its vertices. IDs start at 0 in reverse topological order: an edge between components goes to a smaller ID. num[u] is the DFS entry time; low[u] is the smallest entry reachable while active.

Usage: SCC scc(adj); int id = scc.comp[u]; int count = sz(scc.components);
Time: $\mathcal{O}(N + E)$ time and memory.

3f98cc, 41 lines

```
struct SCC {
    int timer = 0;
    vector<int> num, low, comp, st;
    vector<bool> inStack;
    vector<vector<int>>> components;
    SCC(const vector<vector<int>>> &adj, int firstVertex = 1) {
        int n = sz(adj);
        num.assign(n, 0);
        low.assign(n, 0);
        comp.assign(n, -1);
        inStack.assign(n, false);
        for (int u = firstVertex; u < n; u++)
            if (num[u] == 0) dfs(adj, u);
    }
    void dfs(const vector<vector<int>>> &adj, int u) {
        timer++;
        num[u] = timer;
        low[u] = timer;
        st.push_back(u);
        inStack[u] = true;
        for (int v : adj[u]) {
            if (num[v] == 0) {
                dfs(adj, v);
                low[u] = min(low[u], low[v]);
            } else if (inStack[v]) {
                low[u] = min(low[u], num[v]);
            }
        }
        if (low[u] != num[u]) return;
        int id = sz(components);
        components.push_back({});
        while (true) {
            int v = st.back();
            st.pop_back();
            inStack[v] = false;
            comp[v] = id;
            components[id].push_back(v);
            if (v == u) break;
        }
    }
};
```

594528, 34 lines

```
vector<int> num, st;
vector<vector<pii>>> ed;
int Time;
template <class F>
int dfs(int at, int par, F &f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at])
        if (e != par) {
            if (num[y]) {
                top = min(top, num[y]);
```

```
            if (num[y] < me)
                st.push_back(e);
            } else {
                int si = sz(st);
                int up = dfs(y, e, f);
                top = min(top, up);
                if (up == me) {
                    st.push_back(e);
                    f(vector<int>(st.begin() + si, st.end()));
                    st.resize(si);
                } else if (up < me)
                    st.push_back(e);
                else { /* e is a bridge */
                }
            }
        }
        return top;
    }
};
template <class F>
void bicomps(F f) {
    num.assign(sz(ed), 0);
    for (int i = 0; i < (sz(ed)); ++i)
        if (!num[i]) dfs(i, -1, f);
}
```

2-SAT

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type (a||b)&&(!a||c)&&(d||b)&&... becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

d748cf, 40 lines

```
"scc.h"
struct TwoSat {
    int N;
    vector<vector<int>>> gr;
    vector<int> values; // 0 = false, 1 = true
    TwoSat(int n = 0) : N(n), gr(2 * n) {}
    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }
    void either(int f, int j) {
        f = max(2 * f, -1 - 2 * f);
        j = max(2 * j, -1 - 2 * j);
        gr[f].push_back(j ^ 1);
        gr[j].push_back(f ^ 1);
    }
    void setValue(int x) { either(x, x); }
    void atMostOne(const vector<int> &li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        for (int i = 2; i < (sz(li)); ++i) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
};
```

```
bool solve() {
    SCC scc(gr, 0);
    values.assign(N, 0);
    for (int i = 0; i < N; i++) {
        if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;
        // Node 2*i means false; node 2*i+1 means true.
        values[i] = scc.comp[2 * i] > scc.comp[2 * i + 1];
    }
    return true;
};
```

Euler Walk

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

cdbd50, 22 lines

```
vector<int> eulerWalk(vector<vector<pii>>> &gr, int nedges, int src = 0) {
    int n = sz(gr);
    vector<int> D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D)
        if (x < 0 || sz(ret) != nedges + 1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

Edge Coloring

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

ebd87b, 31 lines

```
vector<int> edgeColoring(int N, vector<pii> eds) {
    vector<int> cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vector<int>>> adj(N, vector<int>(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
```

```
int left = fan[i], right = fan[++i], e = cc[i];
adj[u][e] = left;
adj[left][e] = u;
adj[right][e] = -1;
free[right] = e;
}
adj[u][d] = fan[i];
adj[fan[i]][d] = u;
for (int y : {fan[0], u, end})
    for (int &z = free[y] = 0; adj[y][z] != -1; z++);
}
for (int i = 0; i < (sz(eds)); ++i)
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
return ret;
}
```

7.6 Heuristics

Maximal Cliques

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

d3c296, 17 lines

```
typedef bitset<128> B;
template <class F>
void cliques(vector<B> &eds, F f, B P = ~B(), B X = {}, B R = {}
) {
    if (!P.any()) {
        if (!X.any()) f(R);
        return;
    }
    auto q = (P | X)._Find_first();
    auto cand = P & ~eds[q];
    for (int i = 0; i < (sz(eds)); ++i)
        if (cand[i]) {
            R[i] = 1;
            cliques(eds, f, P & eds[i], X & eds[i], R);
            R[i] = P[i] = 0;
            X[i] = 1;
        }
}
```

Maximum Clique

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

c71c6d, 58 lines

```
typedef vector<bitset<200>> vb;
struct MaximumClique {
    double limit = 0.025, pk = 0;
    struct Vertex {
        int i, d = 0;
    };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vector<int>>> C;
    vector<int> qmax, q, S, old;
    void init(vv &r) {
        for (auto &v : r) v.d = 0;
        for (auto &v : r)
            for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        for (int i = 0; i < (sz(r)); ++i) r[i].d = min(i, mxD) + 1;
    }
}
```

```
void expand(vv &R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
        if (sz(q) + R.back().d <= sz(qmax)) return;
        q.push_back(R.back().i);
        vv T;
        for (auto v : R)
            if (e[R.back().i][v.i]) T.push_back({v.i});
        if (sz(T)) {
            if (S[lev]++ / ++pk < limit) init(T);
            int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
            C[1].clear(), C[2].clear();
            for (auto v : T) {
                int k = 1;
                auto f = [&](int i) { return e[v.i][i]; };
                while (any_of(all(C[k]), f)) k++;
                if (k > mxk) mxk = k, C[mxk + 1].clear();
                if (k < mnk) T[j++].i = v.i;
                C[k].push_back(v.i);
            }
            if (j > 0) T[j - 1].d = 0;
            for (int k = mnk; k < (mxk + 1); ++k)
                for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
            expand(T, lev + 1);
        } else if (sz(q) > sz(qmax))
            qmax = q;
        q.pop_back(), R.pop_back();
    }
}
vector<int> maxClique() {
    init(V), expand(V);
    return qmax;
}
MaximumClique(vb conn) : e(conn), C(sz(e) + 1), S(sz(C)), old
(S) {
    for (int i = 0; i < (sz(e)); ++i) V.push_back({i});
}
};
```

Maximum Independent Set

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

Binary Lifting

Description: Ancestor table up[u][k] stores the ancestor of u after 2^k steps. parent[root] must equal root; parent and depth must use the same vertex indices. Supports either 0-based vectors or 1-based vectors with index 0 reserved. goUp requires $0 \leq \text{steps} \leq \text{depth}[u]$.

Usage: auto up = buildAncestorTable(parent); int p = goUp(up,u,3); int w = lca(up,h,u,v);

Time: $\mathcal{O}(N \log N)$ construction and memory; $\mathcal{O}(\log N)$ per query.

ba907e, 27 lines

```
vector<vector<int>>> buildAncestorTable(const vector<int> &
parent) {
    int n = sz(parent);
    int LOG = 1;
    while ((1LL << LOG) <= n) LOG++;
    vector<vector<int>>> up(n, vector<int>(LOG));
    for (int u = 0; u < n; u++) up[u][0] = parent[u];
    for (int k = 1; k < LOG; k++)
        for (int u = 0; u < n; u++) up[u][k] = up[up[u][k - 1]][k - 1];
    return up;
}
```

```
}
int goUp(const vector<vector<int>>> &up, int u, int steps) {
    for (int k = 0; k < sz(up[u]); k++)
        if ((steps >> k) & 1) u = up[u][k];
    return u;
}
int lca(const vector<vector<int>>> &up, const vector<int> &h,
int u, int v) {
    if (h[u] < h[v]) swap(u, v);
    u = goUp(up, u, h[u] - h[v]);
    if (u == v) return u;
    for (int k = sz(up[u]) - 1; k >= 0; k--) {
        if (up[u][k] != up[v][k]) {
            u = up[u][k];
            v = up[v][k];
        }
    }
    return up[u][0];
}
```

Lowest Common Ancestor (LCA)

Description: Binary-lifting LCA using up[u][k] and depth h[u]. adj is an undirected tree or directed parent-to-child tree. Default root is 1; pass root=0 for a 0-based tree. time[u] is the DFS entry order for virtual trees. distance returns the number of edges, not the sum of edge weights.

Usage: LCA tree(adj); int w = tree.lca(u,v); int p = tree.goUp(u,steps);

Time: $\mathcal{O}(N \log N)$ construction and memory; $\mathcal{O}(\log N)$ per query.

"BinaryLifting.h" b492d3, 30 lines

```
struct LCA {
    int LOG = 1;
    int timer = 0;
    vector<vector<int>>> up;
    vector<int> h, time;
    LCA(const vector<vector<int>>> &adj, int root = 1) {
        int n = sz(adj);
        while ((1LL << LOG) <= n) LOG++;
        up.assign(n, vector<int>(LOG));
        h.assign(n, 0);
        time.assign(n, -1);
        if (root >= 0 && root < n) dfs(adj, root, root);
    }
    void dfs(const vector<vector<int>>> &adj, int u, int parent) {
        time[u] = timer++;
        up[u][0] = parent;
        for (int k = 1; k < LOG; k++) up[u][k] = up[up[u][k - 1]][k - 1];
        for (int v : adj[u]) {
            if (v == parent) continue;
            h[v] = h[u] + 1;
            dfs(adj, v, u);
        }
    }
    int goUp(int u, int steps) const { return ::goUp(up, u, steps
); }
    int lca(int u, int v) const { return ::lca(up, h, u, v); }
    int distance(int u, int v) const {
        int w = lca(u, v);
        return h[u] + h[v] - 2 * h[w];
    }
};
```

Virtual Tree Construction

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig-index) representing a tree rooted at virtual index 0. The root points to itself.

```
Time:  $\mathcal{O}(|S|(\log|S| + \log N))$ 
"LCA.h" 210176, 23 lines

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA &lca, const vector<int> &subset) {
    if (subset.empty()) return {};
    vector<int> rev;
    rev.resize(sz(lca.time));
    vector<int> li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li) - 1;
    for (int i = 0; i < (m); ++i) {
        int a = li[i], b = li[i + 1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    for (int i = 0; i < (sz(li)); ++i) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    for (int i = 0; i < (sz(li) - 1); ++i) {
        int a = li[i], b = li[i + 1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

Heavy-Light Decomposition (HLD)
Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Usage: HLD<false> hld(adj); hld.tree.set(1,sz(adj),0);
Internal positions are 0-based; segment-tree calls convert to inclusive 1-based ranges.
Time: $\mathcal{O}((\log N)^2)$

```
"../data-structures/LazySegmentTree.h" 3c19d8, 46 lines

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vector<int>> adj;
    vector<int> par, siz, rt, pos;
    LazySegTree tree;
    HLD(vector<vector<int>> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1), rt(N),
          pos(N), tree(N) {
        dfsSz(0);
        dfsHld(0);
    }
    void dfsSz(int v) {
        for (int &u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; v = par[rt[v]]) {
```

```
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree.add(l + 1, r, val); });
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int res = INT_MIN;
        process(u, v, [&](int l, int r) { res = max(res, tree.query(l + 1, r)); });
        return res;
    }
    int querySubtree(int v) { // modifySubtree is similar
        return tree.query(pos[v] + VALS_EDGES + 1, pos[v] + siz[v]);
    }
};
```

Directed MST (Arborescence)
Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

```
"../data-structures/UnionFindRollback.h" ab4034, 68 lines

struct ArborescenceEdge {
    int a, b;
    ll w;
};

struct SkewHeapNode {
    ArborescenceEdge key;
    SkewHeapNode *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    ArborescenceEdge top() {
        prop();
        return key;
    }
};

SkewHeapNode *merge(SkewHeapNode *a, SkewHeapNode *b) {
    if (!a || !b) return a ?: b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}

void pop(SkewHeapNode *&a) {
    a->prop();
    a = merge(a->l, a->r);
}

pair<ll, vector<int>> dmst(int n, int r, vector<
    ArborescenceEdge> &g) {
    RollbackDSU uf(n);
    vector<SkewHeapNode *> heap(n);
    for (ArborescenceEdge e : g) heap[e.b] = merge(heap[e.b], new
        SkewHeapNode{e});
    ll res = 0;
    vector<int> seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<ArborescenceEdge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<ArborescenceEdge>>> cys;
```

```
    for (int s = 0; s < (n); ++s) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1, {}};
            ArborescenceEdge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                SkewHeapNode *cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cys.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        for (int i = 0; i < (qi); ++i) in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto &[u, t, comp] : cys) { // restore sol (optional)
        uf.rollback(t);
        ArborescenceEdge inEdge = in[u];
        for (auto &e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    for (int i = 0; i < (n); ++i) par[i] = in[i].a;
    return {res, par};
}
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat, and for each edge $a \rightarrow b \in G$, do mat[a][b]--, mat[b][b]++ (and mat[b][a]--, mat[a][a]++ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

2D Point and Vector

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) { return (x > 0) - (x < 0); }
template <class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
```

```
bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }

bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }

P operator+(P p) const { return P(x + p.x, y + p.y); }
P operator-(P p) const { return P(x - p.x, y - p.y); }
P operator*(T d) const { return P(x * d, y * d); }
P operator/(T d) const { return P(x / d, y / d); }
T dot(P p) const { return x * p.x + y * p.y; }
T cross(P p) const { return x * p.y - y * p.x; }
T cross(P a, P b) const { return (a - *this).cross(b - *this); }

T dist2() const { return x * x + y * y; }
double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
P unit() const { return *this / dist(); } // makes dist()==1
P perp() const { return P(-y, x); } // rotates +90 degrees

P normal() const { return perp().unit(); }
// returns point rotated 'a' radians ccw around the origin
P rotate(double a) const {
    return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
}

friend ostream &operator<<(ostream &os, P p) {
    return os << "(" << p.x << ", " << p.y << ")";
}

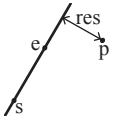
};
```

Point-to-Line Distance

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

```
"Point.h"
53630a, 4 lines

template <class P>
double lineDist(P a, P b, P p) {
    return (double)(b - a).cross(p - a) / (b - a).dist();
}
```



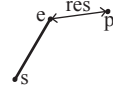
Point-to-Segment Distance

Description:
Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

```
"Point.h"
d5f39f, 6 lines

typedef Point<double> P;
double segDist(P s, P e, P p) {
    if (s == e) return (p - s).dist();
    auto d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```



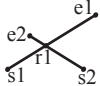
Segment Intersection

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

```
"Point.h", "OnSegment.h"
9d57f2, 14 lines

template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```



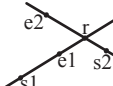
Line Intersection

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

```
"Point.h"
a01f81, 8 lines

template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```



Orientation Test (Cross Product)

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"
ca4665, 8 lines

template <class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
template <class P>
int sideOf(P s, P e, P p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

Point on Segment

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p) <= epsilon) instead when using Point<double>.

```
"Point.h"
c597e8, 4 lines

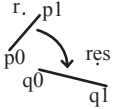
template <class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

Linear Transformation

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
"Point.h"
7871e9, 5 lines

typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
    P dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r - p0).cross(num), (r - p0).dot(num)) / dp.dist2();
}
```



Angle Ordering

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; for (int i = 0; i < (n); ++i) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

```
0f0602, 34 lines

struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t = 0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x - b.x, y - b.y, t}; }
    ; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};

bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x < make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between // them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ? make_pair(a, b) : make_pair(b, a.t360()));
}

Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t;
    a.t = b.t;
    return {a.x * b.x + a.y * b.y, a.x * b.y - a.y * b.x, tu - (b < a)};
}
```

```
}

```

8.2 Circles

Circle Intersection

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"	84d6d3, 14 lines
<pre>typedef Point<double> P; bool circleInter(P a, P b, double r1, double r2, pair<P, P> * out) { if (a == b) { assert(r1 != r2); return false; } P vec = b - a; double d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2, p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 * r1 - p * p * d2; if (sum * sum < d2 dif * dif > d2) return false; P mid = a + vec * p, per = vec.perp() * sqrt(fmax(0, h2) / d2); *out = {mid + per, mid - per}; return true; }</pre>	

Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"	b0153d, 13 lines
<pre>template <class P> vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) { P d = c2 - c1; double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr; if (d2 == 0 h2 < 0) return {}; vector<pair<P, P>> out; for (double sign : {-1, 1}) { P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2; out.push_back({c1 + v * r1, c2 + v * r2}); } if (h2 == 0) out.pop_back(); return out; }</pre>	

Circle-Polygon Intersection

Description: Returns the area of the intersection of a circle with a ccw polygon.

"Time: $\mathcal{O}(n)$ "	
"../././content/geometry/Point.h"	783300, 19 lines
<pre>typedef Point<double> P; #define arg(p, q) atan2(p.cross(q), p.dot(q)) double circlePoly(P c, double r, vector<P> ps) { auto tri = [&](P p, P q) { auto r2 = r * r / 2; P d = q - p; auto a = d.dot(p) / d.dist2(), b = (p.dist2() - r * r) / d. dist2(); auto det = a * a - b; if (det <= 0) return arg(p, q) * r2; auto s = max(0., -a - sqrt(det)), t = min(1., -a + sqrt(det)); if (t < 0 1 <= s) return arg(p, q) * r2;</pre>	

<pre> P u = p + d * s, v = q + d * (t - 1); return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2; }; auto sum = 0.0; for (int i = 0; i < (sz(ps)); ++i) sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c); return sum; }</pre>	
---	--

Circumcircle

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h"	6cffc5, 9 lines
<pre>typedef Point<double> P; double ccRadius(P A, P B, P C) { return (B - A).dist() * (C - B).dist() * (A - C).dist() / abs((B - A).cross(C - A)) / 2; } P ccCenter(P A, P B, P C) { P b = C - A, c = B - A; return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2; }</pre>	

Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points.

"Time: expected $\mathcal{O}(n)$ "	
"circumcircle.h"	eedf5e, 20 lines
<pre>pair<P, double> mec(vector<P> ps) { shuffle(all(ps), mt19937(time(0))); P o = ps[0]; double r = 0, EPS = 1 + 1e-8; for (int i = 0; i < (sz(ps)); ++i) if ((o - ps[i]).dist() > r * EPS) { o = ps[i], r = 0; for (int j = 0; j < (i); ++j) if ((o - ps[j]).dist() > r * EPS) { o = (ps[i] + ps[j]) / 2; r = (o - ps[i]).dist(); for (int k = 0; k < (j); ++k) if ((o - ps[k]).dist() > r * EPS) { o = ccCenter(ps[i], ps[j], ps[k]); r = (o - ps[i]).dist(); } } } return {o, r}; }</pre>	

8.3 Polygons

Point in Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};	
bool in = inPolygon(v, P{3, 3}, false);	
Time: $\mathcal{O}(n)$	
"Point.h", "OnSegment.h", "SegmentDistance.h"	8bc512, 11 lines
<pre>template <class P> bool inPolygon(vector<P> &p, P a, bool strict = true) { int cnt = 0, n = sz(p); for (int i = 0; i < (n); ++i) { P q = p[(i + 1) % n];</pre>	

<pre> if (onSegment(p[i], q, a)) return !strict; //or: if (segDist(p[i], q, a) <= eps) return !strict; cnt ^= ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q) > 0; } return cnt; }</pre>	
---	--

Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	ee890c, 6 lines
<pre>template <class T> T polygonArea2(vector<Point<T>> &v) { T a = v.back().cross(v[0]); for (int i = 0; i < (sz(v) - 1); ++i) a += v[i].cross(v[i + 1]); return a; }</pre>	

Polygon Center

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$	
"Point.h"	9706dc, 10 lines
<pre>typedef Point<double> P; P polygonCenter(const vector<P> &v) { P res(0, 0); double A = 0; for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) { res = res + (v[i] + v[j]) * v[j].cross(v[i]); A += v[j].cross(v[i]); } return res / A / 3; }</pre>	

Polygon Cut

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0,0), P(1,0));

"Point.h"	9b55ae, 13 lines
<pre>typedef Point<double> P; vector<P> polygonCut(const vector<P> &poly, P s, P e) { vector<P> res; for (int i = 0; i < (sz(poly)); ++i) { P cur = poly[i], prev = i ? poly[i - 1] : poly.back(); auto a = s.cross(e, cur), b = s.cross(e, prev); if ((a < 0) != (b < 0)) res.push_back(cur + (prev - cur) * (a / (a - b))); if (a < 0) res.push_back(cur); } return res; }</pre>	

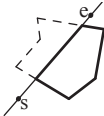
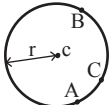
Convex Hull

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
<pre>typedef Point<ll> P; vector<P> convexHull(vector<P> pts) { if (sz(pts) <= 1) return pts;</pre>	



```
sort(all(pts));
vector<P> h(sz(pts) + 1);
int s = 0, t = 0;
for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
        while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0) t
            --;
        h[t++] = p;
    }
return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

Convex Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h"	85d897, 12 lines
-----------	------------------

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    for (int i = 0; i < (j); ++i)
        for (; j = (j + 1) % n; {
            res = max(res, {{S[i] - S[j]}.dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

Point in Convex Polygon

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"	71446b, 13 lines
--------------------------------------	------------------

```
typedef Point<ll> P;
bool inHull(const vector<P> &l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

Line-Convex Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h"	f41089, 41 lines
-----------	------------------

```
#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n])
)
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P> &poly, P dir) {
```

```
int n = sz(poly), lo = 0, hi = n;
if (extr(0)) return 0;
while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
}
return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P> &poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    for (int i = 0; i < (2); ++i) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0:
                return {res[0], res[0]};
            case 2:
                return {res[1], res[1]};
        }
    return res;
}
```

8.4 Misc. Point Set Problems

Closest Pair

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
-----------	------------------

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d(1 + (ll)sqrt(ret.first), 0);
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(p - *lo).dist2(), {p, *lo}});
        S.insert(p);
    }
    return ret.second;
}
```

KD-Tree Nearest Neighbour

Description: KD-tree (2d, can be extended to 3d)
Time: $\mathcal{O}(n \log n)$

"Point.h"	349a63, 60 lines
-----------	------------------

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();
```

```
bool on_x(P a, P b) { return a.x < b.x; }
bool on_y(P a, P b) { return a.y < b.y; }
struct KNode {
    P pt; // if this is a
        leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    KNode *first = 0, *second = 0;
    T distance(P p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1
            : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1
            : p.y);
        return (P(x, y) - p).dist2();
    }
    KNode(vector<P> &vp) : pt(vp[0]) {
        for (P p : vp) {
            x0 = min(x0, p.x);
            x1 = max(x1, p.x);
            y0 = min(y0, p.y);
            y1 = max(y1, p.y);
        }
        if (vp.size() > 1) {
            // split on x if width >= height (not ideal...)
            sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
            // divide by taking half the array for each child (not
            // best performance with many duplicates in the middle)
            int half = sz(vp) / 2;
            first = new KNode({vp.begin(), vp.begin() + half});
            second = new KNode({vp.begin() + half, vp.end()});
        }
    };
};
struct KDTree {
    KNode *root;
    KDTree(const vector<P> &vp) : root(new KNode({all(vp)})) {}
    pair<T, P> search(KNode *node, P p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
            // if (p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        KNode *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }
    // find nearest point to a point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, P> nearest(P p) {
        return search(root, p);
    }
};
```

Strings (9)

Knuth-Morris-Pratt (KMP)

Description: prefixFunction[i] is the longest proper prefix matching a suffix of s[0..i]. kmpSearch returns 0-based start positions of all matches, including overlaps. Empty pattern matches all positions 0..text.size(). Works without a separator character.
Usage: vector<int> positions = kmpSearch("ababa", "aba"); // {0, 2}

Time: $\mathcal{O}(text + pattern)$ time; $\mathcal{O}(pattern)$ auxiliary memory.	7127d4, 32 lines
<pre>vector<int> prefixFunction(const string &s) { int n = sz(s); vector<int> prefix(n, 0); for (int i = 1; i < n; i++) { int matched = prefix[i - 1]; // Fall back to the next shorter prefix until the new // character fits. while (matched > 0 && s[i] != s[matched]) matched = prefix[matched - 1]; if (s[i] == s[matched]) matched++; prefix[i] = matched; } return prefix; } vector<int> kmpSearch(const string &text, const string &pattern) { vector<int> positions; int m = sz(pattern); if (m == 0) { for (int i = 0; i <= sz(text); i++) positions.push_back(i); return positions; } vector<int> prefix = prefixFunction(pattern); int matched = 0; for (int i = 0; i < sz(text); i++) { while (matched > 0 && text[i] != pattern[matched]) matched = prefix[matched - 1]; if (text[i] == pattern[matched]) matched++; if (matched == m) { positions.push_back(i - m + 1); // Keep the longest border so overlapping matches are // found. matched = prefix[matched - 1]; } } return positions; }</pre>	

Z-Function

Description: $z[i]$ is the common-prefix length of s and the suffix starting at i . Positions are 0-based; $z[0] = 0$. Example: abacaba gives 0,0,1,0,3,0,1.
Usage: `vector<int> z = zFunction(s);`
Time: $\mathcal{O}(N)$ time and memory.

	509c9f, 16 lines
<pre>vector<int> zFunction(const string &s) { int n = sz(s); vector<int> z(n, 0); int l = 0; int r = -1; for (int i = 1; i < n; i++) { // [l,r] is the rightmost segment known to match a prefix // of s. if (i <= r) z[i] = min(r - i + 1, z[i - 1]); while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++; if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; } } return z; }</pre>	

Zfunc Manacher MinRotation SuffixArray Hashing

Manacher Palindromes

Description: For each position in a string, computes $p[0][i]$ = half length of longest even palindrome around pos i , $p[1][i]$ = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

	da86e3, 14 lines
<pre>array<vector<int>, 2> manacher(const string &s) { int n = sz(s); array<vector<int>, 2> p = {vector<int>(n + 1), vector<int>(n) }; for (int z = 0; z < (2); ++z) for (int i = 0, l = 0, r = 0; i < n; i++) { int t = r - i + !z; if (i < r) p[z][i] = min(t, p[z][l + t]); int L = i - p[z][i], R = i + p[z][i] - !z; while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1]) p[z][i]++, L--, R++; if (R > r) l = L, r = R; } return p; }</pre>	

Minimum String Rotation

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

	4167a0, 16 lines
<pre>int minRotation(string s) { int a = 0, N = sz(s); s += s; for (int b = 0; b < (N); ++b) for (int k = 0; k < (N); ++k) { if (a + k == b s[a + k] < s[b + k]) { b += max(0, k - 1); break; } if (s[a + k] > s[b + k]) { a = b; break; } } return a; }</pre>	

Suffix Array

Description: Builds suffix array for a string. $sa[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and $sa[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $lcp[i] = lcp(sa[i], sa[i - 1])$, $lcp[0] = 0$. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

	f1087b, 23 lines
<pre>struct SuffixArray { vector<int> sa, lcp; SuffixArray(string s, int lim = 256) { // or vector<int> s.push_back(0); int n = sz(s), k = 0, a, b; vector<int> x(all(s)), y(n), ws(max(n, lim)); sa = lcp = y, iota(all(sa), 0); for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) { p = j, iota(all(y), n - j); for (int i = 0; i < (n); ++i) if (sa[i] >= j) y[p++] = sa[i] - j; fill(all(ws), 0); for (int i = 0; i < (n); ++i) ws[x[i]]++; for (int i = 1; i < (lim); ++i) ws[i] += ws[i - 1]; for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i]; swap(x, y), p = 1, x[sa[0]] = 0; } } };</pre>	

	75d24b, 47 lines
<pre>for (int i = 1; i < (n); ++i) a = sa[i - 1], b = sa[i], x [b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++; } for (int i = 0, j; i < n - 1; lcp[x[i++]] = k) for (k &&k--, j = sa[x[i] - 1]; s[i + k] == s[j + k]; k++); } };</pre>	

Rolling Hash

	75d24b, 47 lines
Description: Self-explanatory methods for string hashing. <pre>// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more // code, but works on evil test data (e.g. Thue-Morse, where // ABBA... and BAAB... of length 2^10 hash the same mod 2^64). // "typedef ull HashValue;" instead if you think test data is // random, // or work mod 10^9+7 if the Birthday paradox is not a problem. struct HashValue { ull x; HashValue(ull x = 0) : x(x) {} HashValue operator+(HashValue o) { return x + o.x + (x + o.x < x); } HashValue operator-(HashValue o) { return *this + ~o.x; } HashValue operator*(HashValue o) { auto m = (__uint128_t)x * o.x; return HashValue((ull)m) + (ull)(m >> 64); } ull get() const { return x + !~x; } bool operator==(HashValue o) const { return get() == o.get(); } bool operator<(HashValue o) const { return get() < o.get(); } }; static const HashValue C = (11)1e11 + 3; // (order ~ 3e9; random also ok) struct RollingHash { vector<HashValue> ha, pw; RollingHash(string &str) : ha(sz(str) + 1), pw(ha) { pw[0] = 1; for (int i = 0; i < (sz(str)); ++i) ha[i + 1] = ha[i] * C + str[i], pw[i + 1] = pw[i] * C; } HashValue hashInterval(int a, int b) { // hash [a, b) return ha[b] - ha[a] * pw[b - a]; } }; vector<HashValue> getHashes(string &str, int length) { if (sz(str) < length) return {}; HashValue h = 0, pw = 1; for (int i = 0; i < (length); ++i) h = h * C + str[i], pw = pw * C; vector<HashValue> ret = {h}; for (int i = length; i < (sz(str)); ++i) { ret.push_back(h = h * C + str[i] - pw * str[i - length]); } return ret; } HashValue hashString(string &s) { HashValue h{}; for (char c : s) h = h * C + c; return h; }</pre>	

Aho-Corasick String Matching

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

<pre>struct AhoCorasick { enum { alpha = 26, first = 'A' }; // change this! struct Node { // (nmatches is optional) int back, next[alpha], start = -1, end = -1, nmatches = 0; Node(int v) { memset(next, v, sizeof(next)); } }; vector<Node> N; vector<int> backp; void insert(string &s, int j) { assert(!s.empty()); int n = 0; for (char c : s) { int &m = N[n].next[c - first]; if (m == -1) { n = m = sz(N); N.emplace_back(-1); } else n = m; } if (N[n].end == -1) N[n].start = j; backp.push_back(N[n].end); N[n].end = j; N[n].nmatches++; } AhoCorasick(vector<string> &pat) : N(1, -1) { for (int i = 0; i < (sz(pat)); ++i) insert(pat[i], i); N[0].back = sz(N); N.emplace_back(0); queue<int> q; for (q.push(0); !q.empty(); q.pop()) { int n = q.front(), prev = N[n].back; for (int i = 0; i < (alpha); ++i) { int &ed = N[n].next[i], y = N[prev].next[i]; if (ed == -1) ed = y; else { N[ed].back = y; (N[ed].end == -1 ? N[ed].end : backp[N[ed].start]) = N[y].end; N[ed].nmatches += N[y].nmatches; q.push(ed); } } } } vector<int> find(string word) { int n = 0; vector<int> res; // ll count = 0; for (char c : word) { n = N[n].next[c - first]; res.push_back(N[n].end); // count += N[n].nmatches; } return res; } };</pre>	7dc5dc, 70 lines
---	------------------

<pre>} vector<vector<int>> findAll(vector<string> &pat, string word) { vector<int> r = find(word); vector<vector<int>> res(sz(word)); for (int i = 0; i < (sz(word)); ++i) { int ind = r[i]; while (ind != -1) { res[i - sz(pat[ind]) + 1].push_back(ind); ind = backp[ind]; } } return res; } };</pre>	
--	--

Various (10)

10.1 Intervals

Interval Union and Removal

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$	edce47, 24 lines
<pre>set<pii>::iterator addInterval(set<pii> &is, int L, int R) { if (L == R) return is.end(); auto it = is.lower_bound({L, R}), before = it; while (it != is.end() && it->first <= R) { R = max(R, it->second); before = it = is.erase(it); } if (it != is.begin() && (--it)->second >= L) { L = min(L, it->first); R = max(R, it->second); is.erase(it); } return is.insert(before, {L, R}); } void removeInterval(set<pii> &is, int L, int R) { if (L == R) return; auto it = addInterval(is, L, R); auto r2 = it->second; if (it->first == L) is.erase(it); else (int &)it->second = L; if (R != r2) is.emplace(R, r2); }</pre>	

Minimum Interval Cover

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$	3f8b99, 19 lines
<pre>template <class T> vector<int> cover(pair<T, T> G, vector<pair<T, T>> I) { vector<int> S(sz(I)), R; iota(all(S), 0); sort(all(S), [&](int a, int b) { return I[a] < I[b]; }); T cur = G.first; int at = 0; while (cur < G.second) { // (A) pair<T, int> mx = make_pair(cur, -1); while (at < sz(I) && I[S[at]].first <= cur) {</pre>	

<pre> mx = max(mx, make_pair(I[S[at]].second, S[at])); at++; } if (mx.second == -1) return {}; cur = mx.first; R.push_back(mx.second); } return R; }</pre>	
---	--

Constant-Value Interval Search

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});

Time: $\mathcal{O}(k \log \frac{n}{k})$	753a4c, 21 lines
<pre>template <class F, class G, class T> void rec(int from, int to, F &f, G &g, int &i, T &p, T q) { if (p == q) return; if (from == to) { g(i, to, p); i = to; p = q; } else { int mid = (from + to) >> 1; rec(from, mid, f, g, i, p, f(mid)); rec(mid + 1, to, f, g, i, p, q); } } template <class F, class G> void constantIntervals(int from, int to, F f, G g) { if (to <= from) return; int i = from; auto p = f(i), q = f(to - 1); rec(from, to - 1, f, g, i, p, q); g(i, to, q); }</pre>	

10.2 Misc. algorithms

Ternary Search

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).

Usage: int ind = ternSearch(0,n-1,[&](int i){return a[i];});

Time: $\mathcal{O}(\log(b - a))$	fea740, 14 lines
<pre>template <class F> int ternSearch(int a, int b, F f) { assert(a <= b); while (b - a >= 5) { int mid = (a + b) / 2; if (f(mid) < f(mid + 1)) a = mid; // (A) else b = mid + 1; } for (int i = a + 1; i < (b + 1); ++i) if (f(a) < f(i)) a = i; // (B) return a; }</pre>	

Fast Subset Sum (Knapsack)

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

6bb4b5, 17 lines

```
int knapsack(vector<int> w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t) a += w[b++];
    if (b == sz(w)) return a;
    int m = *max_element(all(w));
    vector<int> u, v(2 * m, -1);
    v[a + m - t] = b;
    for (int i = b; i < (sz(w)); ++i) {
        u = v;
        for (int x = 0; x < (m); ++x) v[x + w[i]] = max(v[x + w[i]], u[x]);
        for (x = 2 * m; --x > m; )
            for (int j = max(0, u[x]); j < (v[x]); ++j)
                v[x - w[j]] = max(v[x - w[j]], j);
    }
    for (a = t; v[a + m - t] < 0; a--);
    return a;
}
```

10.3 Dynamic programming

Knuth DP Optimization

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j - 1]$ and $p[i + 1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: CHT (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

Divide-and-Conquer DP

Description: Given $a[i] = \min_{l \circ(i) \leq k < h i(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$

b89ccb, 17 lines

```
struct DivideConquerDP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        for (int k = max(LO, lo(mid)); k < (min(HI, hi(mid))); ++k)
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second + 1);
        rec(mid + 1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });`
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in x.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of m (except m itself).
- `c = x&-x, r = x+c; ((r^x) >> 2)/c) | r` is the next number after x with the same number of bits set.
- Computes all sums of subsets:
`for (int b = 0; b < K; ++b)`
`for (int i = 0; i < (1 << K); ++i)`
`if (i & 1 << b)`
`D[i] += D[i^(1 << b)];`

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

Fast Modular Reduction

Description: Compute $a \% b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

38ea39, 7 lines

```
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(~1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

Fast Input

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as `cin/scanf`.

7b3c70, 16 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}
int readInt() {
    int a, c;
```

```
while ((a = gc()) < 40);
if (a == '-') return -readInt();
while ((c = gc()) >= 48) a = a * 10 + c - 480;
return a - 48;
}
```

Techniques (A)

Technique Checklist

158 lines	
Recursion	Pigeon-hole principle
Divide and conquer	Inclusion/exclusion
Finding interesting points in N log N	Catalan number
Algorithm analysis	Pick’s theorem
Master theorem	Number theory
Amortized time complexity	Integer parts
Greedy algorithm	Divisibility
Scheduling	Euclidean algorithm
Max contiguous subvector sum	Modular arithmetic
Invariants	* Modular multiplication
Huffman encoding	* Modular inverses
Graph theory	* Modular exponentiation by squaring
Dynamic graphs (extra book-keeping)	Chinese remainder theorem
Breadth first search	Fermat’s little theorem
Depth first search	Euler’s theorem
* Normal trees / DFS trees	Phi function
Dijkstra’s algorithm	Frobenius number
MST: Prim’s algorithm	Quadratic reciprocity
Bellman-Ford	Pollard-Rho
Konig’s theorem and vertex cover	Miller-Rabin
Min-cost max flow	Hensel lifting
Lovasz toggle	Vieta root jumping
Matrix tree theorem	Game theory
Maximal matching, general graphs	Combinatorial games
Hopcroft-Karp	Game trees
Hall’s marriage theorem	Mini-max
Graphical sequences	Nim
Euler cycles	Games on graphs
Flow networks	Games on graphs with loops
* Augmenting paths	Grundy numbers
* Edmonds-Karp	Bipartite games without repetition
Bipartite matching	General games without repetition
Min. path cover	Alpha-beta pruning
Topological sorting	Probability theory
Strongly connected components	Optimization
2-SAT	Binary search
Cut vertices, cut-edges and biconnected components	Ternary search
Edge coloring	Unimodality and convex functions
* Trees	Binary search on derivative
Vertex coloring	Numerical methods
* Bipartite graphs (=> trees)	Numeric integration
* 3^n (special case of set cover)	Newton’s method
Diameter and centroid	Root-finding with binary/ternary search
K’t h shortest path	Golden section search
Shortest cycle	Matrices
Dynamic programming	Gaussian elimination
Knapsack	Exponentiation by squaring
Coin change	Sorting
Longest common subsequence	Radix sort
Longest increasing subsequence	Geometry
Number of paths in a dag	Coordinates and vectors
Shortest path in a dag	* Cross product
Dynprog over intervals	* Scalar product
Dynprog over subsets	Convex hull
Dynprog over probabilities	Polygon cut
Dynprog over trees	Closest pair
3^n set cover	Coordinate-compression
Divide and conquer	Quadtrees
Knuth optimization	KD-trees
Convex hull optimizations	All segment-segment intersection
RMQ (sparse table a.k.a 2^k-jumps)	Sweeping
Bitonic cycle	Discretization (convert to events and sweep)
Log partitioning (loop over most restricted)	Angle sweeping
Combinatorics	Line sweeping
Computation of binomial coefficients	Discrete second derivatives
	Strings
	Longest common substring
	Palindrome subsequences
	Knuth-Morris-Pratt
	Tries
	Rolling polynomial hashes
	Suffix array
	Suffix tree
	Aho-Corasick
	Manacher’s algorithm
	Letter position lists
	Combinatorial search
	Meet in the middle
	Brute-force with pruning
	Best-first (A*)
	Bidirectional search
	Iterative deepening DFS / A*
	Data structures
	LCA (2^k-jumps in trees in general)
	Pull/push-technique on trees
	Heavy-light decomposition
	Centroid decomposition
	Lazy propagation
	Self-balancing trees
	Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
	Monotone queues / monotone stacks / sliding queues
	Sliding queue using 2 stacks
	Persistent segment tree